



Programming for AI

Lab 04 Understanding the Principles of Object-Oriented Programming (OOP)

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Fall 2026

AIMS AND OBJECTIVES

1. Understand the concept of Object-Oriented Programming.
2. Create classes in Python.
3. Create and use objects.
4. Implement constructors using `__init__()`.
5. Understand encapsulation and access control.
6. Implement inheritance between classes.
7. Understand method overriding.
8. Implement polymorphism in Python

INTRODUCTION

In this lab we will be covering the following topics

Object-Oriented Programming (OOP) is a programming approach that organizes software around objects, which contain data and functions that operate on that data. It helps make programs more organized, reusable, flexible, and easier to maintain. OOP is based on key concepts such as classes, objects, encapsulation, inheritance, and polymorphism, which allow developers to model real-world entities and their relationships in a structured way.

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING

In Python, object-oriented Programming (OOP) is a programming paradigm that uses objects and classes in programming. It aims to implement real-world entities like inheritance, polymorphisms, encapsulation, etc. in the programming. The main concept of OOPs is to bind the data and the functions that work on that together as a single unit so that no other part of the code can access this data.

Pillars of OOP

- Encapsulation
- Polymorphism
- Inheritance
- Abstraction

CLASSES

A class is a blueprint or template for creating objects. It defines the attributes (data) and methods (behavior) that objects created from the class will have.

A class is declared using the class keyword.

```
class Student:
    name = "Ali"
    age = 20

    def display_info(self):
        print("Name:", self.name)
        print("Age:", self.age)
```

OBJECT

An object is an instance of a class. Objects are created from a class and can access the attributes and methods defined inside that class

```
student1 = Student("Ali", 20)
student2 = Student("Sara", 21)

student1.display_info()
student2.display_info()
```

CONSTRUCTOR

A constructor is a special method that is automatically executed when an object is created. In Python, the constructor is written as `__init__()`. It is commonly used to initialize object attributes.

```
class Student:

    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
```

INSTANCE VARIABLES VS CLASS VARIABLES

1. Instance variables are variables that belong to a specific object. Each object can have different values for its instance variables. They are usually defined using `self`.
2. A class variable is a variable that belongs to the class rather than to a particular object. Its value is shared among all objects of the class.

```
class Student:

    university = "ABC University"

    def __init__(self, name):
        self.name = name
```

```
student1 = Student("Ali")
```

```
student2 = Student("Sara")

print(student1.university)
print(student2.university)
```

VARIABLES BASED ON ACCESSIBILITY

1. A public variable can be accessed directly from inside and outside the class. In Python, variables without an underscore prefix are normally considered public.
2. A protected variable is written with a single underscore (_) before its name. It is intended to be used within the class and its subclasses. Python does not strictly prevent access from outside the class; the underscore is mainly a convention indicating that the variable is for internal use.
3. A private variable is written using two underscores (__) before its name. It is intended to be accessed only within the class. Python applies name mangling to private variables, making direct access from outside the class difficult.

```
class Student:

    def __init__(self):
        self.name = "Ali"      # Public variable
        self._marks = 85      # Protected variable
        self.__password = "12345" # Private variable

    def display(self):
        print("Name:", self.name)
        print("Marks:", self._marks)
        print("Password:", self.__password)

student = Student()

# Accessing public variable
print("Public:", student.name)

# Accessing protected variable
print("Protected:", student._marks)

# Private variable is accessed through a class method
student.display()
```

ENCAPSULATION

Encapsulation is the concept of combining data and methods inside a class while controlling access to the internal data. Private variables and methods such as getters and setters can be used to protect data from inappropriate modification.

```
class BankAccount:

    def __init__(self, owner, balance):
        self.owner = owner
```

```

        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")

    def get_balance(self):
        return self.__balance

account = BankAccount("Ali", 5000)

account.deposit(2000)
account.withdraw(1000)

print("Balance:", account.get_balance())

```

GETTER AND SETTER

A getter is a method used to retrieve the value of a private variable, while a setter is used to modify its value. Setters can also be used to validate data before changing it.

```

class Student:

    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks")

student = Student("Ali", 80)

print("Marks:", student.get_marks())

student.set_marks(90)

```

```
print("Updated Marks:", student.get_marks())
```

INHERITANCE

Inheritance allows a child class to acquire attributes and methods from a parent class. It promotes code reuse and allows related classes to share common functionality.

```
class Person:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_person(self):
        print("Name:", self.name)
        print("Age:", self.age)

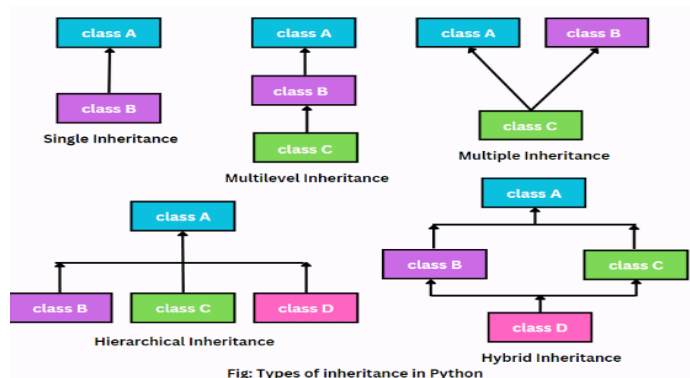
class Student(Person):

    def display_student(self):
        self.display_person()
        print("Student")

student = Student("Ali", 20)
```

```
student.display_student()
```

Python supports the following major types of inheritance:



- **Single Inheritance** – One child class inherits from one parent class.

```
# Parent class
class Person:

    def person_info(self):
        print("This is Person")
```

```
class Student(Person):

    def student_info(self):
        print("This is Student")

student = Student()
student.person_info()
student.student_info()
```

2. Multiple Inheritance – One child class inherits from multiple parent classes.

```
class Father:

    def father_skill(self):
        print("Father: Driving")

class Mother:

    def mother_skill(self):
        print("Mother: Cooking")

class Child(Father, Mother):

    def child_skill(self):
        print("Child: Programming")

child = Child()

child.father_skill()
child.mother_skill()
child.child_skill()
```

3. Multilevel Inheritance – A class inherits from a child class, forming a chain.

```
class GraduateStudent(Student):

    def graduate_info(self):
        print("This is Graduate Student")

graduate = GraduateStudent()

graduate.person_info()
graduate.student_info()
graduate.graduate_info()
```

4. Hierarchical Inheritance – Multiple child classes inherit from the same parent class.

```
class Teacher(Person):
```

```

def teacher_info(self):
    print("This is Teacher")

class Employee(Person):

    def employee_info(self):
        print("This is Employee")

teacher = Teacher()
employee = Employee()

teacher.person_info()
teacher.teacher_info()

employee.person_info()
employee.employee_info()

```

5. Hybrid Inheritance – A combination of two or more types of inheritance.

```

class Person2:

    def person_info(self):
        print("Person")

class Student2(Person2):

    def student_info(self):
        print("Student")

class Teacher2(Person2):

    def teacher_info(self):
        print("Teacher")

class TeachingAssistant(Student2, Teacher2):

    def assistant_info(self):
        print("Teaching Assistant")

assistant = TeachingAssistant()

assistant.person_info()
assistant.student_info()
assistant.teacher_info()

```

POLYMORPHISM

Polymorphism is an important concept of Object-Oriented Programming (OOP) that means “many forms.” It allows the same method name or interface to perform different actions depending on the object that calls it. Polymorphism makes programs more flexible, reusable, and easier to extend.

In Python, polymorphism can commonly be achieved through method overriding, where different child classes provide their own implementation of the same method.

```
class Animal:

    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):

    def sound(self):
        print("Dog barks")

class Cat(Animal):

    def sound(self):
        print("Cat meows")

class Cow(Animal):

    def sound(self):
        print("Cow moos")

# Creating objects
dog = Dog()
cat = Cat()
cow = Cow()

# Same method, different behavior
dog.sound()
cat.sound()
cow.sound()
```

METHOD OVERRIDING

Method overriding occurs when a child class provides its own implementation of a method that is already defined in its parent class. It allows a child class to change or customize the behavior inherited from the parent class. Method overriding is one of the common ways to achieve polymorphism in Python.


```

class Animal:

    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):

    def sound(self):
        print("Dog barks")

class Cat(Animal):

    def sound(self):
        print("Cat meows")

# Creating objects
animal = Animal()
dog = Dog()
cat = Cat()

# Calling the same method
animal.sound()
dog.sound()
cat.sound()

```

ABSTRACTION

Abstraction is the process of hiding unnecessary implementation details and showing only the essential features of an object. It allows programmers to focus on what an object does rather than how it does it. In Python, abstraction can be implemented using abstract classes and abstract methods from the abc module.

```

from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

class Circle(Shape):

    def __init__(self, radius):
        self.radius = radius

    def area(self):

```

```

        return 3.14 * self.radius * self.radius

class Rectangle(Shape):

    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

# Creating objects
circle = Circle(5)
rectangle = Rectangle(10, 4)

print("Circle Area:", circle.area())
print("Rectangle Area:", rectangle.area())

```

LAB TASKS

You are developing a basic Cybersecurity Threat Detection System for a computer network. The system monitors different devices and determines their security status based on the detected threat level. Design and implement a Python class named ThreatDetector.

- The class should contain the following attributes (device_name, ip_address, threat_level).
- Create a method named scan() that displays the security status of the device.
- The system should classify the device according to the following rules:

Threat Level	System Response
Low	System Safe
Medium	Suspicious Activity
High	Critical Threat Detected

- You are developing a Secure Password Vault for a computer system. The password must be protected from direct access while the user should still be able to verify and change it through appropriate methods. Design and implement a Python class named PasswordVault.
- The class should contain: (username as a **public variable**, vault_status as a **protected variable**, password as a **private variable**)
- Implement the following methods: [change_password(),verify_password(),display_status()]
- The verify_password() method should check whether the entered password matches the stored password.
- If the password is correct, display: "Access Granted" , Otherwise, display: Access Denied
- A cybersecurity company is developing a system that uses different security tools to respond to cyber attacks. Although every security tool performs a different action, all of them should provide a common respond() method.Design this system using inheritance and polymorphism.

- Create a parent class named: SecuritySystem , with a method: respond()
 - Create the following child classes:[Firewall, Antivirus, IntrusionDetectionSystem]
 - Each child class must override the respond() method.
 - Their responses should be different. For example:
 - Firewall → Block suspicious network traffic
 - Antivirus → Isolate malicious files
 - ntrusion Detection System → Generate security alert
- You are designing a control system for different types of autonomous robots. Each robot performs a different type of movement, but all robots share common properties such as name and battery level.Design and implement this system using OOP.
- Create a parent class named:Robot
 - The class should contain:[name, battery]
 - Create the following methods:[move(), charge()]
 - Create three child classes:
 - DeliveryRobot
 - SecurityRobot
 - RescueRobot
 - Each child class should provide its own implementation of the move() method.
 - Delivery Robot → moves to a delivery location
 - Security Robot → patrols a specific area
 - Rescue Robot → moves toward a disaster location

Constraint : A robot should not be allowed to move when its battery level is below **20%**.

- You are developing a simple AI Agent System in which different intelligent agents perform different tasks.Although the agents have different responsibilities, the system should interact with all agents through a common method called perform_task().
- Create a parent class named:Agent
- with the following attributes: [name , status]
- with following method:perform_task()
- Create three child classes:
 1. SecurityAgent
 2. MonitoringAgent
 3. RecoveryAgent
- Each child class should override perform_task() and perform a different task.
 1. SecurityAgent → Detecting cyber threat
 2. MonitoringAgent → Monitoring system activity
 3. RecoveryAgent → Recovering system services

6. You are developing a Smart Computer Resource Manager that monitors the resources of a computer and automatically determines whether the system is operating normally or requires attention.Design and implement a Python class named Computer.

- The class should store: [CPU usage , RAM usage, Battery level]
- Create a method named: system_status()
- The method should analyze the resource values and generate appropriate warnings.
- Use the following conditions:

Condition	Warning
CPU > 80%	Heavy CPU Load
RAM > 85%	High Memory Usage
Battery < 20%	Low Battery

20%

- Create at least two computer objects with different resource values.

CONSTRAINTS : If more than one condition is true, the program should display all applicable warnings.

For example, if CPU is 90% and battery is 15%, the program should report both:

Heavy CPU Load
Low Battery

7. You are working as a software developer for a cybersecurity company. The company wants to develop a prototype of an Autonomous Cyber Defense System. The system contains multiple intelligent cyber agents. Each agent analyzes security events and performs an appropriate response. Design and implement this system using OOP principles.

- Create a parent class named: CyberAgent
- The class should contain: [Agent name, Status, **A** private threat score]
- Implement appropriate methods to update and retrieve the threat score.
- Create the following child classes:
 - NetworkAgent
 - MalwareAgent
 - IncidentResponseAgent
- Each child class should implement its own versions of: analyze() ,respond()